

Лабораторна робота №3.

Обмін даними між завданнями за допомогою черг у FreeRTOS

Мета роботи. Навчитися використовувати черги для обміну даними між завданнями.

Теоретичні відомості.

Черга (Queue) - це структура даних у FreeRTOS, яка дозволяє передавати повідомлення (значення або структури) між завданнями. Виклики блокуються, якщо черга порожня (при читанні) або повна (при записі).

Створення черги:

```
QueueHandle_t xQueue = xQueueCreate(довжина, розмір_елемента);
```

Відправлення даних:

```
xQueueSend(xQueue, &дані, таймаут);
```

Отримання даних:

```
xQueueReceive(xQueue, &змінна, таймаут);
```

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Реалізуйте чергу розміром 5 елементів. Передавайте подію "натискання кнопки" у завдання керування LED.

```
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "FreeRTOS.h"
4  #include "task.h"
5  #include "queue.h"
6
7  QueueHandle_t xQueue;
8
9  void vTaskButton(void *pvParameters) {
10     while (1) {
11         bool pressed = gpio_get(3); // кнопка на GPIO3
12         if (pressed) {
13             int msg = 1;
14             xQueueSend(xQueue, &msg, portMAX_DELAY);
15         }
16         vTaskDelay(pdMS_TO_TICKS(200));
```

```

17     }
18 }
19
20 void vTaskLed(void *pvParameters) {
21     int received;
22     while (1) {
23         if (xQueueReceive(xQueue, &received, portMAX_DELAY)) {
24             gpio_put(10, !gpio_get(10)); // toggle LED
25             printf("Button pressed -> LED toggled\n");
26         }
27     }
28 }
29
30 int main() {
31     stdio_init_all();
32     gpio_init(10); gpio_set_dir(10, true);
33     gpio_init(3); gpio_set_dir(3, false); gpio_pull_up(3);
34
35     xQueue = xQueueCreate(5, sizeof(int));
36     xTaskCreate(vTaskButton, "Button", 256, NULL, 1, NULL);
37     xTaskCreate(vTaskLed, "LED", 256, NULL, 1, NULL);
38     vTaskStartScheduler();
39     while (1) {}
40 }

```

3. Модифікуйте код так, щоб прибрати дрижання контактів кнопки.
4. Створіть два завдання: завдання vTaskSensor() - читає дані з потенціометра, завдання vTaskLedControl() - отримує значення з черги і змінює яскравість світлодіода.

```

1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "hardware/adc.h"
4  #include "hardware/pwm.h"
5  #include "FreeRTOS.h"
6  #include "task.h"
7  #include "queue.h"
8
9  QueueHandle_t sensorQueue;
10
11 void vTaskSensor(void *pvParameters) {
12     uint16_t value;
13     while (1) {
14         value = adc_read(); // зчитуємо з АЦП

```

```

15     xQueueSend(sensorQueue, &value, 0);
16     vTaskDelay(pdMS_TO_TICKS(200));
17 }
18 }
19
20 void vTaskLedControl(void *pvParameters) {
21     uint16_t sensorValue;
22     while (1) {
23         if (xQueueReceive(sensorQueue, &sensorValue, portMAX_DELAY)) {
24             uint16_t duty = sensorValue * 16; // масштабування
25             pwm_set_gpio_level(10, duty);
26             printf("ADC: %u -> PWM: %u\n", sensorValue, duty);
27         }
28     }
29 }
30
31 int main() {
32     stdio_init_all();
33     adc_init();
34     adc_gpio_init(27);
35     adc_select_input(1); // канал 1 = GPIO27
36
37     gpio_set_function(10, GPIO_FUNC_PWM);
38     uint slice = pwm_gpio_to_slice_num(10);
39     pwm_set_wrap(slice, 65535);
40     pwm_set_enabled(slice, true);
41
42     sensorQueue = xQueueCreate(10, sizeof(uint16_t));
43
44     xTaskCreate(vTaskSensor, "Sensor", 512, NULL, 2, NULL);
45     xTaskCreate(vTaskLedControl, "LedCtrl", 512, NULL, 1, NULL);
46
47     vTaskStartScheduler();
48     while (1) {}
49 }

```

5. Проаналізуйте поведінку програми:

- Як часто надходять нові значення?
- Що відбувається, якщо `vTaskDelay()` у сенсорному завданні зменшити до 50 мс?
- Чи встигає споживач обробляти всі повідомлення?

6. Змініть розмір черги (1, 5, 20). Як це впливає на поведінку системи?

7. Виведіть повідомлення у UART при переповненні черги.